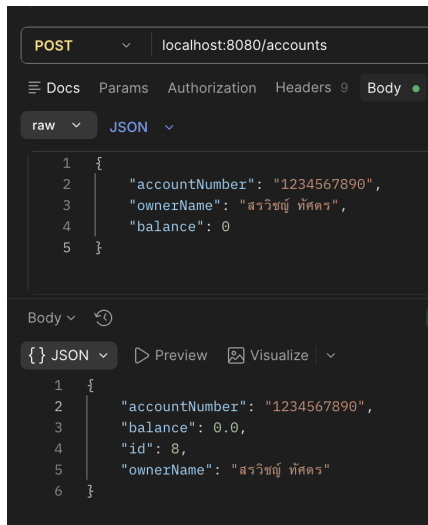


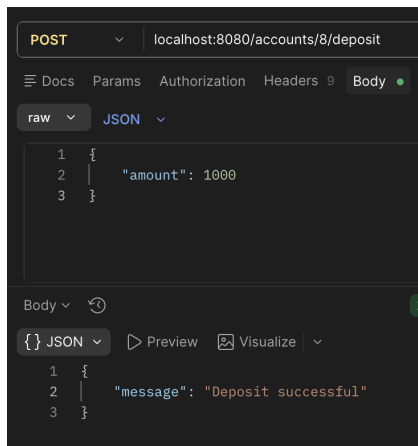
Lab09

1. การสร้าง Account



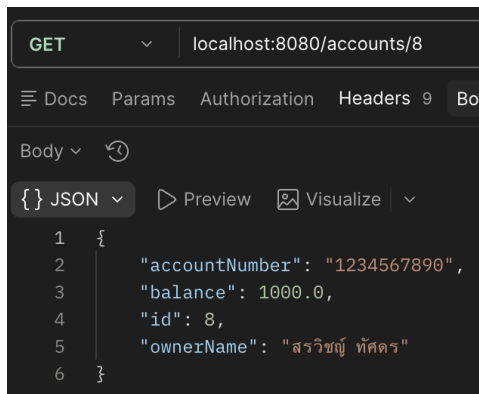
ส่งค่าเข้า model of Account โดยไม่มี null และ id เป็นแบบ auto increment อยู่แล้วเลยไม่ใส่ id in body

2. การฝากเงินสำเร็จ



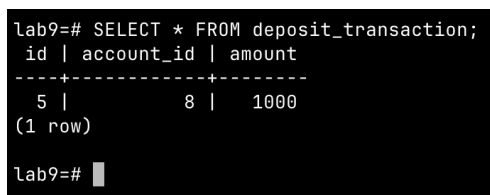
ส่งค่า 1000 เป็นแบบ object แต่เราไม่มี class สำหรับ amount เลยต้องดึงด้วย Map เอา value ออกมาคำนวณ โดยใช้ old balance บวกกับ new balance

3. ตรวจสอบ Account



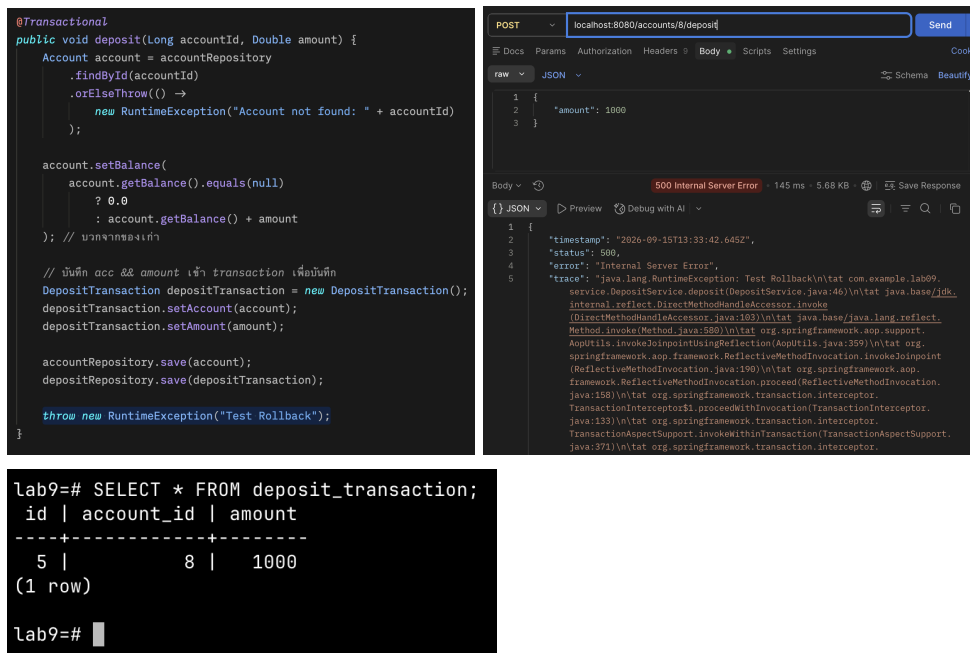
แสดงค่าจาก Account หรือใน sql คือ select * from account;

4. ตรวจสอบ DepositTransaction



query table deposit_transaction ออกมาเฉยๆ

5. ทดลอง Rollback



ทดสอบการ rollback โดยการใส่ throw error ไปในโค้ด และ ก็ไม่บันทึกประวัติอีกด้วย

6. เปรียบเทียบมี @Transactional กับไม่มี @Transactional

```
@Transactional
public void deposit(Long accountId, Double amount) {
    Account account = accountRepository
        .findById(accountId)
        .orElseThrow(() ->
            new RuntimeException("Account not found: " + accountId)
        );

    account.setBalance(
        account.getBalance().equals(null)
            ? 0.0
            : account.getBalance() + amount
    ); // บวกจากของเก่า

    // บันทึก acc && amount เข้า transaction เพื่อบันทึก
    DepositTransaction depositTransaction = new DepositTransaction();
    depositTransaction.setAccount(account);
    depositTransaction.setAmount(amount);

    accountRepository.save(account);
    depositRepository.save(depositTransaction);

    throw new RuntimeException("Test Rollback");
}
```

มี error แต่ database ไม่มีการเปลี่ยนแปลงอะไรทั้งสิ้น

```
lab9=# SELECT * FROM deposit_transaction;
 id | account_id | amount 
----+-----+-----
  5 |          8 |    1000
(1 row)

lab9=#
```

```
// @Transactional
public void deposit(Long accountId, Double amount) {
    Account account = accountRepository
        .findById(accountId)
        .orElseThrow(() ->
            new RuntimeException("Account not found: " + accountId)
        );

    account.setBalance(
        account.getBalance().equals(null)
            ? 0.0
            : account.getBalance() + amount
    ); // บวกจากของเก่า

    // บันทึก acc && amount เข้า transaction เพื่อบันทึก
    DepositTransaction depositTransaction = new DepositTransaction();
    depositTransaction.setAccount(account);
    depositTransaction.setAmount(amount);

    accountRepository.save(account);
    depositRepository.save(depositTransaction);

    throw new RuntimeException("Test Rollback");
}
```

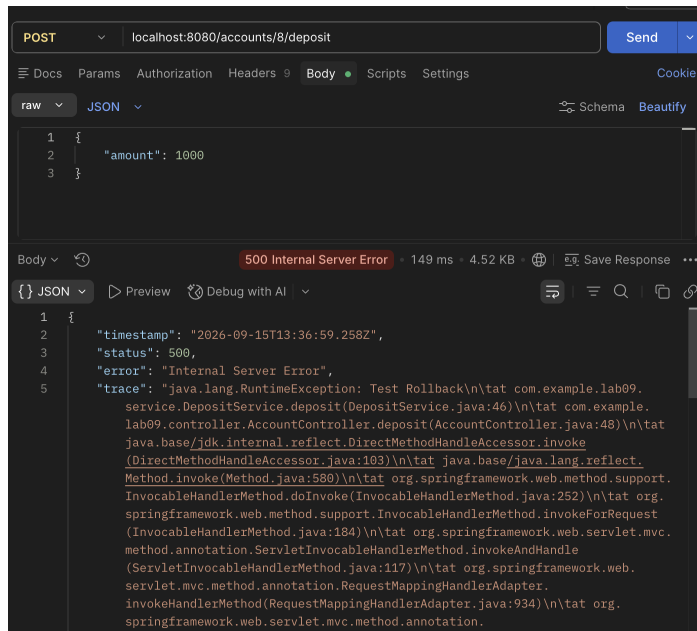
มี error แต่ database เปลี่ยน

```
lab9=# select * from account;
 id | account_number | balance | owner_name 
----+-----+-----+-----
  8 | 1234567890    |    2000 | สรรวิชัย ทัศนธร
(1 row)

lab9=#
```

```
lab9=# SELECT * FROM deposit_transaction;
 id | account_id | amount 
----+-----+-----
  5 |          8 |    1000
  9 |          8 |    1000
(2 rows)
```

โดยที่ทั้งสองอัน error เหมือนกันใน postman (หน้าต่อไป)



7. ตอบคำถามท้าย Lab ทั้ง 6 ข้อ

1. `@Transactional` มีหน้าที่อะไร?
> เช็ครการทำงานว่าสำเร็จทั้งหมดไหมในส่วนนั้น หากสำเร็จจะ commit หากไม่จะ rollback
2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?
> เพื่อไม่ให้เกิด consistency ของระบบ
3. COMMIT และ ROLLBACK ต่างกันอย่างไร?
> commit เป็นการทำงานจากโค้ดที่ทำการลง database
> rollback เป็นการย้อนกลับการทำงานของโค้ดในระบบ
4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?
> Account 1 ----- N DepositTransaction , One to Many
5. เมื่อเกิด Error ขณะใช้ `@Transactional` ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?
> ไม่เปลี่ยน เพราะเกิดการ rollback
6. จากการทดลอง เมื่อลบ `@Transactional` ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?
> เกิด Error แต่ยังไม่บันทึกการทำงานลง database เหมือนเดิม